KeenPay

Agentic commerce with bounded AI

KeenPay Discount Policy Schema (Bounded AI Pattern)

Discount policy schema (DDL)

01 September 2026

https://github.com/ambitiouss22/KeenPay

KeenPay Discount Policy Schema (Bounded AI Pattern)

```
-- Merchant-defined discount bounds preventing unbounded AI proposals.
-- Implements the "bounded AI" pattern: AI proposes ONLY within merchant limits.
--
-- Tables:
--   discount_policies       - Merchant-defined bounds per product
--   discount_segments       - Per-user-type (new, returning, vip, bulk) limits
--   discount_usage_tracking - Daily/weekly budget consumption
--   discount_decisions      - Audit trail of all discount proposals + decisions
--
-- Key Pattern:
--   1. Merchant defines policy (max discount %, daily budget in paise)
--   2. AI proposes discount within bounds
--   3. DiscountPolicyEngine validates requested discount
--   4. Returns approved_discount (may be < requested)
--   5. Every decision logged for audit
--
-- All money in integer paise (1 paise = 1/100 INR).
-- ============================================================================
```

SQL

ENUMS for discount policies

```
CREATE TYPE user_segment AS ENUM (
    'new',
    'returning',
    'vip',
    'bulk_buyer'
);

CREATE TYPE discount_decision_status AS ENUM (
    'APPROVED',
    'REDUCED',
    'DENIED'
);
```

SQL

DISCOUNT POLICIES - Merchant-defined bounds

```
-- One policy per merchant per product SKU.
-- Defines global max, per-segment max, daily budget, blacklisted combos.

CREATE TABLE discount_policies (
    policy_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id         VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    product_sku         VARCHAR(64) NOT NULL,

    -- Global max discount (0.0 - 100.0)
    max_discount_pct    NUMERIC(5, 2) NOT NULL CHECK (max_discount_pct >= 0 AND max_discount_pct <= 100),

    -- Budget limits
    daily_budget_paise  BIGINT NOT NULL CHECK (daily_budget_paise > 0),
    weekly_budget_paise BIGINT NOT NULL DEFAULT 0 CHECK (weekly_budget_paise >= 0),

    -- Blacklisted discount combinations (stored as JSON array of strings)
    -- Example: ["free_shipping+50pct_off", "buy_one_get_one+discount"]
    blacklist_combos    JSONB NOT NULL DEFAULT '[]'::jsonb,

    -- Policy metadata
    is_active           BOOLEAN NOT NULL DEFAULT TRUE,
    description         TEXT,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- Ensure one policy per merchant per SKU
    CONSTRAINT discount_policies_unique UNIQUE (merchant_id, product_sku)
);

COMMENT ON TABLE discount_policies IS 'Merchant-defined discount bounds (bounded AI pattern)';
COMMENT ON COLUMN discount_policies.max_discount_pct IS 'Global maximum across all user types; AI cannot exceed this';
COMMENT ON COLUMN discount_policies.daily_budget_paise IS 'Total discount budget per day; once exhausted, discounts reduced to 0';
COMMENT ON COLUMN discount_policies.weekly_budget_paise IS 'Optional weekly cap; 0 means disabled';
COMMENT ON COLUMN discount_policies.blacklist_combos IS 'Incompatible discount combinations; e.g. free_shipping + 50% off cannot combine';

CREATE INDEX idx_discount_policies_merchant ON discount_policies (merchant_id, is_active);
CREATE INDEX idx_discount_policies_sku ON discount_policies (product_sku) WHERE is_active = TRUE;
CREATE INDEX idx_discount_policies_active ON discount_policies (is_active);
```

SQL

DISCOUNT SEGMENTS - Per-user-type limits

```
-- Overrides global max for specific user segments.
-- Example: VIP users can get up to 25% off even if global max is 15%.

CREATE TABLE discount_segments (
    segment_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_id           UUID NOT NULL REFERENCES discount_policies(policy_id) ON DELETE CASCADE,

    user_segment        user_segment NOT NULL,
    max_discount_pct    NUMERIC(5, 2) NOT NULL CHECK (max_discount_pct >= 0 AND max_discount_pct <= 100),

    description         TEXT,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- One segment per policy
    CONSTRAINT discount_segments_unique UNIQUE (policy_id, user_segment)
);

COMMENT ON TABLE discount_segments IS 'Per-user-type discount limits within a policy';
COMMENT ON COLUMN discount_segments.max_discount_pct IS 'Maximum for this segment; min(global_max, segment_max) is final limit';

CREATE INDEX idx_discount_segments_policy ON discount_segments (policy_id);
```

SQL

DISCOUNT USAGE TRACKING - Budget consumption

```sql
-- Track daily and weekly spending for each policy.
-- Resets daily at midnight UTC, weekly on Monday UTC.

CREATE TABLE discount_usage_tracking (
    tracking_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_id           UUID NOT NULL REFERENCES discount_policies(policy_id) ON DELETE CASCADE,

    tracking_date       DATE NOT NULL,
    tracking_week_start DATE NOT NULL,  -- Monday of the week in UTC

    -- Daily consumption
    daily_used_paise    BIGINT NOT NULL DEFAULT 0 CHECK (daily_used_paise >= 0),
    daily_remaining     BIGINT NOT NULL,  -- Computed: daily_budget_paise - daily_used_paise

    -- Weekly consumption
    weekly_used_paise   BIGINT NOT NULL DEFAULT 0 CHECK (weekly_used_paise >= 0),
    weekly_remaining    BIGINT NOT NULL,

    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- One tracking per policy per date
    CONSTRAINT discount_usage_tracking_unique UNIQUE (policy_id, tracking_date)
);

COMMENT ON TABLE discount_usage_tracking IS 'Budget consumption tracked per day and week for enforcement';
COMMENT ON COLUMN discount_usage_tracking.tracking_date IS 'UTC date for daily reset';
COMMENT ON COLUMN discount_usage_tracking.tracking_week_start IS 'Monday UTC of the week for weekly reset';

CREATE INDEX idx_discount_usage_tracking_policy ON discount_usage_tracking (policy_id);
CREATE INDEX idx_discount_usage_tracking_date ON discount_usage_tracking (tracking_date DESC);
```

SQL

DISCOUNT DECISIONS - Audit trail

```sql
-- Every discount proposal and decision logged for compliance and debugging.
-- Immutable append-only log.

CREATE TABLE discount_decisions (
    decision_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    policy_id           UUID NOT NULL REFERENCES discount_policies(policy_id) ON DELETE RESTRICT,
    session_id          UUID REFERENCES negotiation_sessions(id) ON DELETE SET NULL,

    merchant_id         VARCHAR(64) NOT NULL DEFAULT 'merchant_keen',
    user_id             VARCHAR(64),
    user_segment        user_segment NOT NULL,
    product_sku         VARCHAR(64) NOT NULL,

    -- Proposal
    requested_discount_pct   NUMERIC(5, 2) NOT NULL CHECK (requested_discount_pct >= 0 AND requested_discount_pct <= 100),
    reason_for_proposal      TEXT,  -- Why AI proposed this discount

    -- Decision
    status               discount_decision_status NOT NULL,
    approved_discount_pct    NUMERIC(5, 2) NOT NULL CHECK (approved_discount_pct >= 0 AND approved_discount_pct <= 100),
    decision_reason      TEXT,  -- Explanation of what happened

    -- Which policy rules applied
    policy_applied       VARCHAR(128) NOT NULL,
    -- Examples:
    --   "USER_TYPE_LIMIT_VIP" (user type max exceeded global max)
    --   "GLOBAL_MAX_LIMIT" (global max exceeded)
    --   "DAILY_BUDGET_PARTIAL" (daily budget exhausted, reduced discount)
    --   "DAILY_BUDGET_EXCEEDED" (daily budget exhausted, zero discount)
    --   "BLACKLIST_COMBO_ADJUSTED" (combination not allowed)
    --   "APPROVED_WITHIN_BOUNDS" (all checks passed)

    -- Policy state at decision time
    policy_snapshot     JSONB NOT NULL,  -- Snapshot of policy limits
    usage_snapshot      JSONB NOT NULL,  -- Snapshot of budget usage at time

    -- Timestamps
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

COMMENT ON TABLE discount_decisions IS 'Audit trail of all discount proposals and approvals (append-only)';
COMMENT ON COLUMN discount_decisions.status IS 'APPROVED (matched requested), REDUCED (lower than requested), DENIED (0%)';
COMMENT ON COLUMN discount_decisions.policy_applied IS 'Which rule triggered; used for debugging and analytics';
COMMENT ON COLUMN discount_decisions.policy_snapshot IS 'Snapshot of policy at decision time for replay/audit';

CREATE INDEX idx_discount_decisions_policy ON discount_decisions (policy_id, created_at DESC);
CREATE INDEX idx_discount_decisions_session ON discount_decisions (session_id) WHERE session_id IS NOT NULL;
CREATE INDEX idx_discount_decisions_user ON discount_decisions (user_id, created_at DESC) WHERE user_id IS NOT NULL;
CREATE INDEX idx_discount_decisions_sku ON discount_decisions (product_sku, created_at DESC);
CREATE INDEX idx_discount_decisions_status ON discount_decisions (status, created_at DESC);

-- Append-only enforcement: prevent updates/deletes
CREATE OR REPLACE FUNCTION discount_decisions_deny_mutation()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'discount_decisions is append-only';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_discount_decisions_no_update
    BEFORE UPDATE ON discount_decisions
    FOR EACH ROW EXECUTE FUNCTION discount_decisions_deny_mutation();

CREATE TRIGGER trg_discount_decisions_no_delete
    BEFORE DELETE ON discount_decisions
    FOR EACH ROW EXECUTE FUNCTION discount_decisions_deny_mutation();
```

SQL

EXAMPLE DATA (sample merchant discount policies)

```sql
-- Policy 1: Hoodie product - limited discounts
INSERT INTO discount_policies (merchant_id, product_sku, max_discount_pct, daily_budget_paise, weekly_budget_paise, description, is_active)
VALUES (
    'merchant_keen',
    'HOODIE-NAVY-M',
    25.0,
    50000,    -- 500 INR daily budget
    300000,   -- 3000 INR weekly budget
    'Hoodie Navy M: Up to 25% off, max 500 INR/day'
);

-- Segments for hoodie: VIP gets more
INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct, description)
SELECT policy_id, 'vip', 35.0, 'VIP: up to 35% off (override 25% global)'
FROM discount_policies
WHERE merchant_id = 'merchant_keen' AND product_sku = 'HOODIE-NAVY-M';

INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct, description)
SELECT policy_id, 'bulk_buyer', 30.0, 'Bulk buyers: up to 30% off'
FROM discount_policies
WHERE merchant_id = 'merchant_keen' AND product_sku = 'HOODIE-NAVY-M';

-- Policy 2: Tee product - tight discounts
INSERT INTO discount_policies (merchant_id, product_sku, max_discount_pct, daily_budget_paise, weekly_budget_paise, description, is_active)
VALUES (
    'merchant_keen',
    'TEE-BLACK-M',
    15.0,
    20000,    -- 200 INR daily budget
    100000,   -- 1000 INR weekly budget
    'Tee Black M: Up to 15% off, tight budget'
);
```

SQL

STORED FUNCTIONS for budget management

```sql
-- Reset daily budget for today (call at midnight UTC)
CREATE OR REPLACE FUNCTION reset_daily_discount_budget(p_policy_id UUID, p_today DATE)
RETURNS void AS $$
DECLARE
    v_daily_budget BIGINT;
BEGIN
    SELECT daily_budget_paise INTO v_daily_budget
    FROM discount_policies
    WHERE policy_id = p_policy_id;

    INSERT INTO discount_usage_tracking (
        policy_id, tracking_date, tracking_week_start,
        daily_used_paise, daily_remaining, weekly_used_paise, weekly_remaining
    )
    VALUES (
        p_policy_id, p_today, date_trunc('week', p_today)::DATE,
        0, v_daily_budget, 0,
        COALESCE((SELECT weekly_budget_paise FROM discount_policies WHERE policy_id = p_policy_id), 0)
    )
    ON CONFLICT (policy_id, tracking_date) DO UPDATE
    SET daily_used_paise = 0,
        daily_remaining = EXCLUDED.daily_remaining;
END;
$$ LANGUAGE plpgsql;

-- Reset weekly budget for the week
CREATE OR REPLACE FUNCTION reset_weekly_discount_budget(p_policy_id UUID, p_week_start DATE)
RETURNS void AS $$
DECLARE
    v_weekly_budget BIGINT;
BEGIN
    SELECT weekly_budget_paise INTO v_weekly_budget
    FROM discount_policies
    WHERE policy_id = p_policy_id;

    UPDATE discount_usage_tracking
    SET weekly_used_paise = 0,
        weekly_remaining = v_weekly_budget,
        updated_at = NOW()
    WHERE policy_id = p_policy_id
    AND tracking_week_start = p_week_start;
END;
$$ LANGUAGE plpgsql;
```

SQL

VIEWS for analytics

```sql
-- Current discount policy effectiveness
CREATE OR REPLACE VIEW discount_policy_analytics AS
SELECT
    p.policy_id,
    p.merchant_id,
    p.product_sku,
    p.max_discount_pct,
    p.daily_budget_paise,
    COUNT(DISTINCT dd.decision_id) as total_decisions,
    COALESCE(ROUND(AVG(CASE WHEN dd.status = 'APPROVED' THEN dd.approved_discount_pct ELSE NULL END), 2), 0) as avg_approved_pct,
    COALESCE(SUM(CASE WHEN dd.status = 'APPROVED' THEN 1 ELSE 0 END), 0) as approved_count,
    COALESCE(SUM(CASE WHEN dd.status = 'REDUCED' THEN 1 ELSE 0 END), 0) as reduced_count,
    COALESCE(SUM(CASE WHEN dd.status = 'DENIED' THEN 1 ELSE 0 END), 0) as denied_count,
    MAX(dd.created_at) as last_decision_at
FROM discount_policies p
LEFT JOIN discount_decisions dd ON p.policy_id = dd.policy_id
WHERE p.is_active = TRUE
GROUP BY p.policy_id, p.merchant_id, p.product_sku, p.max_discount_pct, p.daily_budget_paise;

COMMENT ON VIEW discount_policy_analytics IS 'Dashboard view: policy effectiveness and decision patterns';
```

SQL

PRODUCTION CHECKLIST

```
-- [ ] discount_policies and discount_decisions append-only (triggers in place)
-- [ ] All money stored as integer paise
-- [ ] discount_usage_tracking reset daily/weekly via scheduled function
-- [ ] Merchant API for creating/updating discount_policies (gated)
-- [ ] User segments enforced: new, returning, vip, bulk_buyer
-- [ ] DiscountPolicyEngine.check_discount_request() called before AI proposes
-- [ ] Every discount decision logged in discount_decisions table
-- [ ] Dashboard query: discount_policy_analytics view
-- [ ] Monitoring: track_decisions_denied_count, avg_approved_discount_pct
```

SQL